Polymath15, fourth thread: closing in on the test problem

Terence Tao · 24 February 2018

https://terrytao.wordpress.com/2018/02/24/polymath15-fourth-thread-closing-in-on-the-test-problem/

tao2typ · converted to Typst 2026-10-14 (UTC)

This is the fourth "research" thread of the Polymath15 project to upper bound the de Bruijn-Newman constant Λ, continuing https://terrytao.wordpress.com/2018/01/24/polymath-proposal-upper-bounding-the-de-bruijn-newman-constant/. Progress will be summarised at this Polymath wiki page.

We are getting closer to finishing off the following test problem: can one show that $H_t(x+iy) \neq 0$ whenever $t = y = 0.4$, $x \geq 0$? This would morally show that $\Lambda \leq 0.48$. A wiki page for this problem has now been created here. We have obtained a number of approximations $A+B$, $A'+B'$, $A^{eff}+B^{eff}$, $A^{toy}+B^{toy}$ to H_t (see wiki page), though numeric evidence indicates that the approximations are all very close to each other. (Many of these approximations come with a correction term C, but thus far it seems that we may be able to avoid having to use this refinement to the approximations.) The effective approximation $A^{eff}+B^{eff}$ also comes with an effective error bound

$$\left|H_t - A^{eff} - B^{eff}\right| \leq E_1 + E_2 + E_3$$

for some explicit (but somewhat messy) error terms E_1, E_2, E_3: see this wiki page for details. The original approximations $A+B$, $A'+B'$ can be considered deprecated at this point in favour of the (slightly more complicated) approximation $A^{eff}+B^{eff}$; the approximation $A^{toy}+B^{toy}$ is a simplified version of $A^{eff}+B^{eff}$ which is not quite as accurate but might be useful for testing purposes.

It is convenient to normalise everything by an explicit non-zero factor B_0^{eff}. Asymptotically, $(A^{eff}+B^{eff})/B_0^{eff}$ converges to 1; numerically, it appears that its magnitude (and also its real part) stays roughly between 0.4 and 3 in the range $10^5 \leq x \leq 10^6$, and we seem to be able to keep it (or at least the toy counterpart $(A^{toy}+B^{toy})/B_0^{toy}$) away from zero starting from about $x \geq 4 \times 10^6$ (here it seems that there is a useful trick of multiplying by Euler-type factors like $1 - \frac{1}{2^{1-s}}$ to cancel off some of the oscillation). Also, the bounds on the error $(H_t - A^{eff} - B^{eff})/B_0^{eff}$ seem to be of size about 0.1 or better in these ranges also. So we seem to be on track to be able to rigorously eliminate zeroes starting from about $x \geq 10^5$ or so. We have not discussed too much what to do with the small values of x; at some point our effective error bounds will become unusable, and we may have to find some more faster ways to compute H_t.

In addition to this main direction of inquiry, there have been additional discussions on the dynamics of zeroes, and some numerical investigations of the behaviour Lehmer pairs under heat flow. Contributors are welcome to summarise any findings from these discussions from previous threads (or on any other related topic, e.g. improvements in the code) in the comments below.

Comments (105)



Terence Tao · 24 February 2018

A note that some of the phases and exponents in the toy approximation were not quite correct in the previous post, but are now correct on the wiki (here it was very helpful by the way to have numerics for all the other approximations as this was very effective at detecting these inaccuracies). The expression $(A^{toy}+B^{toy})/B_0^{toy}$ now takes the form

$$\sum_{n=1}^{N} \frac{1}{n^{\frac{1+y-ix}{2} - \frac{\pi i t}{8} + \frac{t}{4}\log^2 \frac{N^2}{n}}} + e^{i\theta} N^{-y} \sum_{n=1}^{N} \frac{1}{n^{\frac{1-y+ix}{2} + \frac{\pi i t}{8} + \frac{t}{4}\log^2 \frac{N^2}{n}}}$$

for some explicit but probably not very important phase θ. To make this non-zero it suffices to show that

$$\left|\sum_{n=1}^{N} \frac{1}{n^{\frac{1+y+ix'}{2}+\frac{t}{4}\log^2\frac{N^2}{n}}}\right| > N^{-y}\left|\sum_{n=1}^{N} \frac{1}{n^{\frac{1-y+ix'}{2}+\frac{t}{4}\log^2\frac{N^2}{n}}}\right|$$

where $x' := x + \pi t/4$ is a slight shift of x. This is basically the same problem as before except for a very tiny shift in the x parameter. In particular the trick of multiplying by factors such as $1 - \frac{1}{2^{\frac{1+y+ix'}{2}+\frac{t}{4}\log^2\frac{N^2}{2}}}$ should still be useful (a key point being that this factor also produces a bit of cancellation on the RHS as well as the LHS, albeit to a lesser extent due to the mismatch in the y term of the exponent).



Terence Tao · 24 February 2018

To set some explicit benchmarks, I think a good roadmap would be the following:

Goal 1. Show that $(A^{toy} + B^{toy})/B_0^{toy}$ is non-vanishing for any $x > 0$ (or maybe say $x > 100$ to exclude any pathologies at small x coming from the inaccurate nature of the approximation).

Goal 2. Show that $(A^{eff} + B^{eff})/B_0^{eff}$ is non-vanishing for any $x > 0$ (or $x > 100$, etc.)

Goal 3. Show that for all decently large x, one can lower bound $\left|A^{eff} + B^{eff}\right|/\left|B_0^{eff}\right|$ by $\left|H_t - A^{eff} - B^{eff}\right|/\left|B_0^{eff}\right| \leq (E_1 + E_2 + E_3)/\left|B_0^{eff}\right|$ (this will solve the test problem for sufficiently large x, something additional will have to be done when x is so small that the $A^{eff} + B^{eff}$ approximation is no longer effectively accurate).

My guess is that once we figure out how to efficiently achieve Goal 1, Goal 2 would be doable by a slight modification of the method, and Goal 3 would then also be relatively straightforward (of course we have to become better at controlling the errors E_1, E_2, E_3 for this).

As mentioned above, to establish Goal 1 it suffices to establish the inequality

$$\left|\sum_{n=1}^{N} \frac{1}{n^{0.7+\frac{ix'}{2}+0.1\log\frac{N^2}{n}}}\right| > N^{-0.4}\left|\sum_{n=1}^{N} \frac{1}{n^{0.3+\frac{ix'}{2}+0.1\log\frac{N^2}{n}}}\right|$$

for all x in the range of interest, where $x' = x + \pi/10$ and $N = \left\lfloor\sqrt{x/4\pi}\right\rfloor$. Some thoughts on this:

(a) The variable N keeps jumping with x at square multiples of 4π. However, we may be able to get away with rounding N down to the nearest multiple of (say) 100 and treating the error by the triangle inequality. This creates significantly fewer jumps and we can start using the fundamental theorem of calculus (bounding the derivative of the expressions inside the absolute values) between each pair of jumps. Another minor side benefit of decreasing N is that it may also improve slightly the bound on the derivative (the large n terms are a bit more oscillatory than the small n terms).

(b) As before we can try to multiply both sides by Euler factors like $1 - \frac{1}{2^{0.7+\frac{ix'}{2}+0.1\log\frac{N^2}{2}}}$ to have less oscillation (this may help reduce the upper bound on the derivative).

(c) If we have an upper bound on the derivative then we can try to use that to sample x adaptively, rather than using a worst-case mesh, as this may lead to some useful speedup. For instance, suppose $\left|A^{toy} + B^{toy}/B_0^{toy}\right|$ numerically ranges between 0.4 and 3 and has a derivative that can be provably upper bounded by 5 (these numbers are made up for sake of argument). Then we could be conservative and sample $\left|A^{toy} + B^{toy}/B_0^{toy}\right|$ for x in an equally spaced mesh of size $0.4/5$, and as long as the ratio $\left|A^{toy} + B^{toy}/B_0^{toy}\right|$ always evaluates to be greater than 0.4 on this mesh, we are OK for Goal 1. But if for instance at some value of x this ratio evaluates to be larger, say 2.1, then we can skip ahead to $x + 2.1/5$ rather than $x + 0.4/5$ since that is the first place where a zero could conceivably occur. There are variants of this strategy where we evaluate the first derivative at the mesh point and use some upper bound on the second derivative, together with a Taylor expansion with remainder, to try to get an even larger skip, but perhaps just a first derivative analysis will already give a sufficient speedup for our purposes.

Terence Tao · 24 February 2018

On further thought I think (a) is not going to give significant speedups. The intervals $[4N^2\pi, 4(N+1)^2\pi]$ on which N is constant are so much larger than the mesh sizes we are considering that there is little to gain by concatenating them into even larger intervals, so one may as well treat each such interval separately. Even for x up to 4×10^7, N only goes up to about 1784, and the cost of doing 1784 individual calculations would be trivial compared against the total number of mesh points one has to work with.

The other thing is that one does not have to work on the above goals sequentially; it is also possible for instance for some participants to work on understanding the error terms E_1, E_2, E_3 even before say Goal 1 is completed. Also I am beginning to suspect that the error analysis may not be sufficient to handle medium values of x, such as $x \sim 10^4$, and one may indeed need to revive the "C" term in the $A + B - C$ type approximations in order to obtain a significantly better error term, as suggested in previous comments, to handle this range.

David Bernier (@doubledeckerpot) · 25 February 2018

I thought of multiplying the sumands on the LHS of inequality 2 by:
$1 = n^{-y}/n^{-y}$
to render denominators in sums on LHS and RHS equal (I think).

Anonymous · 25 February 2018

this idea of using equal denominators is also mentioned in https://terrytao.wordpress.com/2018/02/24/polymath15-fourth-thread-closing-in-on-the-test-problem/#comment-493119

David Bernier (@doubledeckerpot) · 24 February 2018

For B_0^{eff}, I get a pretty good agreement with the tables when I take a z with imaginary part 0.4, as shown below:

? B0(10^3 + 0.4*I)
= 3.4426 E-167 + 3.5411 E-167*I
? B0(10^4 + 0.4*I)
= -1.1850 E-1700 − 7.7879 E-1700*I
? B0(10^5 + 0.4*I)
= -7.6133 E-17047 + 2.5066 E-17047*I

So I suppose z is $x + 0.4i$.

Terence Tao · 24 February 2018

Yes, throughout that wiki page it is assumed that $t = y = 0.4$. Thanks for checking against the tables, it is always good to have independent confirmation to guard against errors.

Anonymous · 24 February 2018

Is it possible to exploit also some (perhaps crude) information on the asymptotics of the phases of the dominant parts of $A + B$ and the error term (not merely their magnitudes) to improve on the triangle inequality ?

Terence Tao · 24 February 2018

My feeling is that we will not be able to get much out of this, because A and B oscillate in opposite directions (they are sort of conjugate to each other), and so one will periodically run into situations in which the triangle inequality is best possible. A caricature of the situation would be

if we had $A(x) = e^{i(\frac{x}{4}\log\frac{x}{4\pi}-\frac{x}{4})}a(x)$ and $B(x) = e^{-i(\frac{x}{4}\log\frac{x}{4\pi}-\frac{x}{4})}b(x)$ where $a(x), b(x)$ were fairly slowly varying (note that the toy approximations A^{toy}, B^{toy} are basically of this form, assuming that the small n components of the sum $n = 1, ..., N$ dominate, as they seem to do numerically). If one wanted to keep $A(x) + B(x)$ away from zero, it would suffice to show that $|a(x)| > |b(x)|$ for all x. On the other hand, if there was an interval of moderate length (e.g. a unit interval) on which one instead had $|a(x)| < |b(x)|$, then $A(x) + B(x)$ would wind around the origin a non-trivial number of times, and so by the argument principle one should expect to have a zero of $A(x) + B(x)$ for some $y \geq 0.4$.

Anonymous · 24 February 2018

In the wiki page on the test problem, N is defined in the first approach (and its modification) without the square root.
[Corrected, thanks – T.]

Anonymous · 24 February 2018

Another possibility for fast simulteneous(!) computation of $A + B$, B_0 and the bound on the error term on a large grid, is to use ideas from Platt's paper on a (high precision) FFT-based method (with rigorous bounds).

Terence Tao · 24 February 2018

It does seem that the trick of multiplying by an Euler factor also produces a moderate improvement in the derivative bounds.
Recall that we are trying to prove a bound of the form $|b(x)| > |a(x)|$ for all x, where

$$b(x) := \sum_{n=1}^{N} \frac{1}{n^{0.7+ix'/2+0.1\log(N^2/n)}}$$

$$a(x) := N^{-0.4} \sum_{n=1}^{N} \frac{1}{n^{0.3+ix'/2+0.1\log(N^2/n)}},$$

$x' = x + \pi/10$, $N = \lfloor \sqrt{x/4\pi} \rfloor$. Let us pick a value of N, so that x ranges in the interval $[4N^2\pi, 4(N+1)^2\pi)$. I picked $N = 892$ for sake of discussion, as this gives the interval that contains $x = 10^7$. On this interval, the triangle inequality gives the bounds

$$|b(x) - 1| \leq \sum_{n=2}^{N} \frac{1}{n^{0.7+0.1\log(N^2/n)}} \approx 0.946$$

$$|a(x)| \leq N^{-0.4} \sum_{n=1}^{N} \frac{1}{n^{0.3+0.1\log(N^2/n)}} \approx 0.2789$$

which is not quite enough to make the $n = 1$ term of b dominate since $0.946 + 0.2789 > 1$. We have the derivative bounds

$$|b'(x)| \leq \frac{1}{2} \sum_{n=1}^{N} \frac{\log n}{n^{0.7+0.1\log(N^2/n)}} \approx 1.096$$

$$|a'(x)| \leq \frac{1}{2} N^{-0.4} \sum_{n=1}^{N} \frac{\log n}{n^{0.3+0.1\log(N^2/n)}} \approx 0.4144$$

so the quantity $|b(x)| - |a(x)|$ fluctuates at a rate of at most $1.096 + 0.4144 \approx 1.511$, and one could design a mesh (either fixed-spacing or adaptive) to explore the interval $[4N^2\pi, 4(N+1)^2\pi)$ accordingly.
Now suppose we transform the inequality $|b(x)| > |a(x)|$ to the equivalent inequality $|b_2(x)| > |a_2(x)|$, where

$$b_2(x) = b(x)(1 - \frac{1}{2^{0.7+ix'/2+0.1\log(N^2/2)}})$$

$$a_2(x) = a(x)(1 - \frac{1}{2^{0.7+ix'/2+0.1\log(N^2/2)}}).$$

A brief computation shows that

$$b_2(x) := \sum_{n=1}^{2N} \frac{1_{n\leq N} - 1_{2\mid n}/(n/2)^{0.2\log 2}}{n^{0.7+ix'/2+0.1\log(N^2/n)}}$$
$$a_2(x) := N^{-0.4} \sum_{n=1}^{2N} \frac{1_{n\leq N} - 1_{2\mid n}2^{-0.4}/(n/2)^{0.2\log 2}}{n^{0.3+ix'/2+0.1\log(N^2/n)}}.$$
The same triangle inequality analysis as before gives the bounds
$$|b_2(x) - 1| \leq \sum_{n=2}^{2N} \frac{\left|1_{n\leq N} - 1_{2\mid n}/(n/2)^{0.2\log 2}\right|}{n^{0.7+0.1\log(N^2/n)}} \approx 0.480$$
$$|a_2(x)| \leq N^{-0.4} \sum_{n=1}^{2N} \frac{\left|1_{n\leq N} - 1_{2\mid n}2^{-0.4}/(n/2)^{0.2\log 2}\right|}{n^{0.3+0.1\log(N^2/n)}} \approx 0.2300$$
$$|b_2'(x)| \leq \frac{1}{2} \sum_{n=1}^{2N} \frac{\left|1_{n\leq N} - 1_{2\mid n}/(n/2)^{0.2\log 2}\right| \log n}{n^{0.7+0.1\log(N^2/n)}} \approx 0.7373$$
$$|a_2'(x)| \leq \frac{1}{2} N^{-0.4} \sum_{n=1}^{2N} \frac{\left|1_{n\leq N} - 1_{2\mid n}2^{-0.4}/(n/2)^{0.2\log 2}\right| \log n}{n^{0.3+0.1\log(N^2/n)}} \approx 0.3620$$
so in fact now $|b_2(x)| - |a_2(x)|$ is bounded away from zero (it is at least $1 - 0.480 - 0.2300 \approx 0.2897$) and oscillates at rate at most $0.7373 + 0.3620 \approx 1.0993$, thus we have removed a bit of the oscillation from $|b(x)| - |a(x)|$ by this procedure.

One gets similar improvements for smaller values of N. For instance for $N = 282$ (corresponding to $x = 10^6$), we have $|b(x) - 1| \leq 1.4455$, $|a(x)| \leq 0.6047$, $|b'(x)| \leq 1.7850$, $|a'(x)| \leq 0.8781$ and $|b_2(x) - 1| \leq 0.8074$, $|a_2(x)| \leq 0.5001$, $|b_2'(x)| \leq 1.2605$, $|a_2'(x)| \leq 0.7796$. Thus it should be a lot faster to verify $|b_2(x)| - |a_2(x)| > 0$ than $|b(x)| - |a(x)| > 0$, as the former left-hand side oscillates less rapidly and is also more likely to stay further away from the origin, so one would need fewer points to evaluate the function.

As suggested by KM one might also multiply through by the Euler product at 3 and 5 and perhaps get some further modest gains. (But at this point the complexity of evaluating these more complicated sums may overwhelm the gain coming from having fewer evaluation points.) Another thing one can try is to modify the Euler factors slightly, e.g. considering multiplying by $1 - \frac{\theta}{2^{0.7+ix'/2+0.1\log(N^2/2)}}$ for some parameter θ that is not necessarily equal to 1. Right now I am using $\theta = 1$ as this kills off completely the $n = 2$ term in the $b(x)$ sum, but this might not necessarily be the best choice.



KM · 25 February 2018

I was able to match the above numbers and generate estimates for all N from 1 to 1784 (x upto 4*10^7) upto the first 3 Euler factors.

Interesting N values

————————threshold

estimate————0—0.05–0.1

|b5(x)|-|a5(x)|—282—308—338

|b3(x)|-|a3(x)|—322—352—387

|b2(x)|-|a2(x)|—478—527—582

|b1(x)|-|a1(x)|–1391–1559–1761

N=338 corresponds to x ~ 1.5 million

upper bound of derivatives

————————N range

estimate————1-28—29-63–64-89–90-199–200-282–283-337

max|b'5(x)|+|a'5(x)|—4.05—3.86—3.32—3.04—-2.24—-1.89

max|b'3(x)|+|a'3(x)|—3.44—3.43—3.14—2.92—-2.23—-1.91

max|b'2(x)|+|a'2(x)|—3.10—3.17—3.09—2.93—-2.34—-2.04

max|b'1(x)|+|a'1(x)|—3.61—3.82—3.79—3.66—-3.02—-2.66

N=28 corresponds to x=10568. Assuming |A+B|/|B0| stays over 0.4 beyond N=28, this indicates a mesh size of 0.1 or slightly lower would work at the start.

Also, the derivative bound fluctuates in lower N ranges, and at higher ranges max|b'2(x)|+|a'2(x)| is not very different compared to
max|b'p(x)|+|a'p(x)| with p>2.



Anonymous · 25 February 2018

Some of the formulas in comments have log^2(N^2 / n) and in later comments similar formulas have log(N^2 / n) instead. Is that real or a typo?
[Oops, a typo: it should be single power of log throughout, thanks – T.]



Vsevolod · 25 February 2018

Quite a small typo: "or at least the toy counterpart" instead of "ot at least the toy counterpart"
[Corrected, thanks – T.]



David Bernier (@doubledeckerpot) · 25 February 2018

After correcting a sign error I made in defining B^{eff}, I get excellent agreement between the tables in the wiki and my PARI/gp version of them. Below, I recopy the corrected functions needed to define B^{eff} in Pari/GP:

```
B(Z)=(1/8)*exp((t/4)*conj(al1(I*real(Z)/2+(1+imag(Z))/2))^2)*conj(H01(I*real(Z)/2+(1+imag(Z))/2))*sum(Y=1,M(Z),1/
exp((-I*real(Z)/2+(1+imag(Z))/2+conj(t*al1(I*real(Z)/2+(1+imag(Z))/2))/2-(t/4)*log(Y))*log(Y)))
al1(S)=1/(2*S)+1/(S-1)+(1/2)*log(S/(2*Pi))
H01(S)=(S*(S-1)/2)*exp((-S/2)*log(Pi))*sqrt(2*Pi)*exp((S/2-1/2)*log(S/2)-S/2)
M(Z)=floor(sqrt((real(Z)/2-Pi*t/8)/(2*Pi)))
```



Rudolph · 25 February 2018

David,
Below the Pari/GP code I wrote to compute Ht-effective including all the error terms. Maybe you could use parts of it or compare outcomes with your own code. It already contains the fix of the small sign error in T' and T'_0 that was spotted today (I checked the impact and it is indeed very small).
Overall very good matches with the Wiki data, although small differences (in the 3rd digit) emerge at higher x. On Github, we have compared the output with KM's mpmath program and the only remaining (small) difference seems to reside in the method of computation of the vwf-integral. The vwf-integral limits in the code are currently set at 6. The value of t can be set at the top.

```
\\Ht_effective A + B + Err (=E1 + E2) + C (=E3)
 \\init
 default(realprecision, 20)
 t=0.4
 intlim=6
 S(s)=real(s)
 T(s)=imag(s)
 S1(s)=1-S(s)+T(s)*I
 Td(s)=T(s)+Pi*t/8
 T3(s)=T(s)-3.33
 T4(s)=12*(Td(s)-0.33)
 M(s)=floor(sqrt(Td(s)/(2*Pi)))
\\calculate A and B
 ALF(s)=1/(2*s)+1/(s-1)+1/2*log(s/(2*Pi))
 H00(s)=(s/2)*(s-1)*Pi^(-s/2)
```

```
H01(s)= H00(s)*sqrt(2*Pi)*exp((s-1)/2*log(s/2)-s/2)
QA(s,n)=s+t/2*ALF(s)-t/4*log(n)
QB(s,n)=1-s+t/2*conj(ALF(S1(s)))-t/4*log(n)
A0(s)=exp(t/4*ALF(s)^2)*H01(s)
B0(s)=exp(t/4*conj(ALF(S1(s)))^2)*conj(H01(S1(s)))
A(s)=A0(s)*sum(n=1,M(s),1/(n^(QA(s,n))))
B(s)=B0(s)*sum(n=1,M(s),1/(n^(QB(s,n))))
\\eps error
SE(s,n)=s+t/2*ALF(s)-t/4*log(n)
ER(s)=1/(2*T3(s))*(1/4*t^2*(abs(ALF(s))^2)+1/3+t)
EPS(s)=zeta(real(SE(s,M(s))))*exp(ER(s))*ER(s)
Err(s)=abs(A0(s)*EPS(s))+abs(B0(s)*EPS(S1(s)))
\\C and vwf error
a0(s)=sqrt(Td(s)/(2*Pi))
v1(sig,s)=1+0.4*9^(sig)/a0(s)+0.346*2^(1.5*sig)/(a0(s)^2)
v21(sig,s)=0.9^(ceil(-sig))
v22(sig,s)=sum(k=1,floor(-1*sig)+4,(1.1/a0(s))^k*gamma(k/2))
v2(sig,s)=1+v21(sig,s)*v22(sig,s)
v(sig,s)=if(real(sig)>=0,v1(sig,s),v2(sig,s))
w1(sig,s)=1+(sig^2)/(Td(s)^2)
w2(sig,s)=1+((1-sig)^2)/(Td(s)^2)
w3(sig,s)=(sig-1)*log(w1(sig,s))/4+w4(sig,s)+1/T4(s)
w4(sig,s)=if(sig<0,(Td(s)/2*atan(sig/Td(s))-sig/2),0)
w(sig,s)=sqrt(w1(sig,s))*sqrt(w2(sig,s))*exp(w3(sig,s))
f1(sig,s)=exp((-1/t)*(sig-S(s))^2)
f(sig,s)=(1/(2*sqrt(Pi*t)))*(f1(sig,s)+f1(1-sig,s))
vwf(s)=intnum(u = -intlim, intlim, v(u,s)*w(u,s)*f(u,s))
C1(s)=sqrt(Pi)*exp(-(t*Pi^2)/64)*(Td(s))^(3/2)*exp(-Pi/4*T(s))
C(s)=C1(s)*vwf(s)
\\all components together
E(s)=1/8*(A(s)+B(s)+C(s)+Err(s))
H(z)=(E(I*real(z)/2+(1-imag(z))/2))
\\generate output: note that Ht(x+yi)=E(1/2+(x+y*i)/2*i)
print((H(10000000+0.4*I)))
print((E(1/2+(10000000+0.4*I)/2*I)))
print((1/8*B0(1/2+(10000000+0.4*I)/2*I)))
print((B(1/2+(10000000+0.4*I)/2*I)/B0(1/2+(10000000+0.4*I)/2*I)))
`4.6138 E-1705458 - 1.4530 E-1705457*I
 4.6138 E-1705458 - 1.4530 E-1705457*I
 2.1644 E-1705458 - 9.6337 E-1705458*I
 1.6400 + 0.019799*I
```



Rudolph · 25 February 2018

Oops. Compared the code once again with the Wiki-formulae and spotted a mistake. The C-factor should not be divided by 8, so the correct line should be:

E(s)=1/8*(A(s)+B(s)+Err(s))+C(s)

The 1/8 factor can of course also be applied directly to A(s),B(s) and Err(s), when they are defined.

Fortunately the impact of this error on the final result is small.



Terence Tao · 25 February 2018

Actually, you discovered a typo in the wiki! The 1/8 should in fact be applied to the E_3 error, sorry about that. Fortunately this typo moves in the "good" direction :)



Terence Tao · 25 February 2018

I made a minor sign error in the effective error estimates, but fortunately it is in a good direction: the quantities T' and T'_0, which were previously defined as $T - \pi t/8$ and $T'_0 - \pi t/8$ respectively, are instead $T + \pi t/8$ and $T'_0 + \pi t/8$ (I had the wrong sign for β_N in the estimation of $G_{t,N}$). This slightly changes N in the $A^{eff} + B^{eff}$ approximation, which is now $\lfloor \sqrt{(x + \pi t/4)/4\pi} \rfloor$ rather than $\lfloor \sqrt{(x - \pi t/4)/4\pi} \rfloor$. I found this because the formula for $x' = x + \pi t/4$ in the comments above also had the plus sign instead of the minus one, which looked inconsistent or at least suspicious. I don't think this changes the previous numerics substantially though, given how small $\pi t/4 = 0.314\ldots$ is.



Terence Tao · 25 February 2018

It seems the calculations for the toy problem are clarified using the language of Dirichlet series. Indeed observe that

$$b(x) = \sum_{n=1}^{N} \frac{b_n}{n^s}$$

$$a(x) = \sum_{n=1}^{N} \frac{a_n}{n^s}$$

where $s := 0.7 + ix'/2 + 0.1\log(N^2)$, $b_n := \exp(0.1\log^2 n)$, and $a_n := (n/N)^{0.4} b_n$. The vanilla triangle inequality argument gives

$$|b(x) - 1| \leq \sum_{n=2}^{N} \frac{b_n}{n^\sigma}$$

$$|a(x)| \leq \sum_{n=1}^{N} \frac{a_n}{n^\sigma}$$

where $\sigma := 0.7 + 0.1\log(N^2)$, thus one can keep $|b(x)| > |a(x)|$ so long as

$$\sum_{n=1}^{N} \frac{|b_n| + |a_n|}{n^\sigma} < 2;$$

these are the same bounds as before.

Now the Euler factor trick can be viewed as a special case of the standard analytic number theory trick of mollifying a Dirichlet series by multiplying it by a short Dirichlet polynomial to remove some of the main sources of oscillation. Indeed, one can multiply $b(x)$ and $a(x)$ by the same Dirichlet polynomial $f(x) := \sum_{d=1}^{D} \frac{\lambda_d}{d^s}$ for an arbitrary collection of coefficients λ_d to obtain

$$b(x)f(x) = \sum_{n=1}^{DN} \frac{\sum_{d=1}^{D} 1_{n \leq dN} 1_{d \mid n} \lambda_d b_{n/d}}{n^s}$$

$$a(x)f(x) = \sum_{n=1}^{DN} \frac{\sum_{d=1}^{D} 1_{n \leq dN} 1_{d \mid n} \lambda_d a_{n/d}}{n^s}.$$

If we normalise $\lambda_1 = 1$, the triangle inequality then gives

$$|b(x)f(x) - 1| \leq \sum_{n=2}^{DN} \frac{\left|\sum_{d=1}^{D} 1_{n \leq dN} 1_{d \mid n} \lambda_d b_{n/d}\right|}{n^\sigma}$$

$$|a(x)f(x)| \leq \sum_{n=1}^{DN} \frac{\left|\sum_{d=1}^{D} 1_{n \leq dN} 1_{d \mid n} \lambda_d a_{n/d}\right|}{n^\sigma}$$

and we can now ensure that $|b(x)| > |a(x)|$ so long as

$$\sum_{n=1}^{DN} \frac{\left|\sum_{d=1}^{D} 1_{n \leq dN} 1_{d \mid n} \lambda_d b_{n/d}\right| + \left|\sum_{d=1}^{D} 1_{n \leq dN} 1_{d \mid n} \lambda_d a_{n/d}\right|}{n^\sigma} < 2.$$

The vanilla triangle inequality corresponds to just using $D = 1$ and $\lambda_1 = 1$. Using the Euler factor at 2 corresponds to taking $D = 2$, $\lambda_1 = 1$, and $\lambda_2 = -b_2$ (this cancels the $n = 2$ summand of the b term). This might not be the optimal choice of λ_2 to use in the $D = 2$ case. Using the Euler factor at

both 2 and 3 corresponds to taking $D = 6$, $\lambda_1 = 1$, $\lambda_2 = -b_2$, $\lambda_3 = -b_3$, $\lambda_6 = +b_2 b_3$, and all other λ_d vanishing (incidentally my calculations for this case differ slightly from KM's, though I was able to confirm his calculations for a_1, b_1 and a_2, b_2, I think the contribution of the $d = 6$ term could be a bit tricky, as I had made some errors by hand with this term before). However, one is free to use other values of λ_d, and it could be that this could lead to further ranges of N for which we can guarantee $|b(x)| > |a(x)|$ (thus far we tentatively seem to have reached all the way down to $N \geq 282$).

 **Terence Tao** · 25 February 2018

Here is a lemma that may be useful.

Lemma Let a_n, b_n be real numbers for $n = 1, ..., N$ with $b_1 = 1$ and $-1 < a_1 < 1$. Let σ be a real number, and suppose that

$$\sum_{n=2}^{N} \frac{\max(\frac{|b_n+a_n|}{1+a_1}, \frac{|b_n-a_n|}{1-a_1})}{n^\sigma} < 1.$$

Then one has

$$\left|\sum_{n=1}^{N} \frac{b_n}{n^s}\right| > \left|\sum_{n=1}^{N} \frac{a_n}{n^s}\right|$$

whenever $\mathrm{Re}(s) = \sigma$.

Remark The condition (1) improves a little bit upon the condition

$$|a_1| + \sum_{n=2}^{N} \frac{|b_n| + |a_n|}{n^\sigma} < 1$$

that one gets from directly applying the triangle inequality; the gain is particularly good when a_n, b_n have coherent signs (e.g. they are all positive). Intuitively, the point is that the triangle inequality is only sharp if the b_n/n^s are all negative reals for $n \geq 2$, and the a_n/n^s are all the same phase for $n \geq 1$, and this is inconsistent with all the a_n, b_n having the same sign (with $b_1 = 1$ and a_1 positive).

Proof It suffices to show that for any real phase θ, the quantity

$$\sum_{n=1}^{N} \frac{b_n + e^{i\theta} a_n}{n^s}$$

is non-zero. By the triangle inequality, we can do this if

$$\sum_{n=2}^{N} \frac{|b_n + e^{i\theta} a_n|}{|1 + e^{i\theta} a_1|} \frac{1}{n^\sigma} < 1.$$

Observe from the cosine rule that

$$\frac{|b_n+e^{i\theta}a_n|^2}{|1+e^{i\theta}a_1|^2} = \frac{b_n^2+a_n^2+2a_n b_n \cos\theta}{1+a_1^2+2a_1\cos\theta}$$

is a fractional linear function of $\cos\theta$ with no poles in the range $[-1, 1]$ of $\cos(\theta)$, and is hence monotone on this interval Thus $\frac{|b_n+e^{i\theta}a_n|}{|1+e^{i\theta}a_1|}$ attains its maximum either when $e^{i\theta} = +1$ or $e^{i\theta} = -1$, thus

$$\frac{|b_n+e^{i\theta}a_n|}{|1+e^{i\theta}a_1|} \leq \max(\frac{|b_n+a_n|}{1+a_1}, \frac{|b_n-a_n|}{1-a_1})$$

and the claim follows. □

Using this criterion in place of the triangle inequality, and not using any Euler factor mollifiers, I can improve the threshold $N \geq 1391$ that KM obtained to $N \geq 1080$. Presumably similar gains can be obtained when one combines this lemma with the Euler factors (or with other Dirichlet mollifiers, as I mentioned in the previous comment), but I haven't tried this.

 **KM** · 26 February 2018

Using the sharper inequality, I was able to match N=1080 with no Euler factor, but the results with the factor mollifiers seem aggressive.

Mollifier—N

Euler2——296

Euler3——136

Euler5——80

N= 80 corresponds to 80425!! This may need to be double checked.
where b_p_n = exp(0.1*logn*logn)*|conditional factor_p_n| using the coefficients derived earlier.
Also, I was able to run |b_p(x)| – |a_p(x)| delta evaluations with fixed mesh size 0.1 for all x between 10k and 200k (1.9 mil evaluation points) and p=1,2,3. From earlier data, the upper bound of |b'p(x)| + |a'p(x)| was known to be below 4, so I was looking for events where the delta was below 0.4.
For |b_1(x)| – |a_1(x)|, the last such event was at x=38280 (N=55) for a total of 782 events. Minimum was 0.186 reached at 17291.2
For |b_2(x)| – |a_2(x)|, this occured at x=22024 (N=41) for a total of 152 events. Minimum was 0.185 reached at 10530 (the earlier point was a close second)
For |b_3(x)| – |a_3(x)|, this occured at x=21538.5 (N=41) for a total of 72 events. Minimum was 0.201 reached at 17291.2
Using a mesh size of 0.05, the region x=10k to 22k was again verified, looking for delta values below 0.16 (since max|b'2(x)|+|a'2(x)| in the N range 29-63 was 3.17), but no such values were found, thus clearing all the points.
It would have been better to use a adaptive mesh as suggested earlier, rather than use such a iterative procedure.
Also, you mentioned that arbitrary lambda coefficients are allowed in the mollifiers, but just wanted to share the way I had proceeded with the d=6 derivation earlier

$$\frac{1}{n^{s'-0.1\log(n)}}\frac{-1}{2^{s'-0.1\log(2)}}\frac{-1}{3^{s'-0.1\log(3)}}$$
$$=\frac{1}{(6n)^{s'}}\frac{1}{n^{-0.1\log(n)}2^{-0.1\log(2)}3^{-0.1\log(3)}}$$
$$=\frac{1}{(6n)^{s'}}\frac{1}{(6n)^{-0.1\log(n)}6^{0.1\log(n)}(n^{-0.1\log(6)}n^{0.1\log(6)})(6^{-0.1\log(6)}6^{0.1\log(6)})2^{-0.1\log(2)}3^{-0.1\log(3)}}$$
$$=\frac{1}{(6n)^{s'}}\frac{1}{(6n)^{-0.1\log(6n)}6^{0.1\log(n)}n^{0.1\log(6)}2^{0.1\log(3)}3^{0.1\log(2)}}$$
$$=\frac{1}{(6n)^{s'-0.1\log(6n)}}\frac{1}{n^{0.2\log(6)}exp(0.2\log 2\log(3)}$$

with $s'=0.7+ix/2+0.1\log(N^2)$(and an additional $\frac{1}{6^{0.4}}$ factor in the RHS)
leading to the final conditional factors



Terence Tao · 26 February 2018

Thanks for this! The bounds are looking remarkably good, I will try to double check them with my somewhat primitive Maple skills. It also seems that your calculation of the d=6 term was correct and mine was wrong.
If this all checks out it seems that we actually have a numerical verification of the non-vanishing of $A^{toy}+B^{toy}$ for all $x\geq 10^4$. Next we would have to adapt these methods to $A^{eff}+B^{eff}$. A new technical difficulty here is that there is an expression $\alpha_1(s)$ in the exponent which depends on x and not just on N. However the dependence on x is rather mild and we may be able to somehow approximate $\alpha_1(s)$ by something that only depends on N.
As requested I will try to upload my Maple code to the git repository, though I am not really proficient in these things and I would hardly recommend using my code for much other than confirmation.



KM · 26 February 2018

I was also able to write a general procedure for the triangle inequality estimates, which can use any number of Euler factors (although it's quite slow at the moment). Using 4 factors, the 0 threshold was crossed slightly below N=265. Will try to adapt it for the sharper inequality as well.

Terence Tao · 26 February 2018

KM, so far I have verified the triangle inequality thresholds of $N \geq 1391,478,322$ using $Euler_1, Euler_2, Euler_3$ respectively, and for $N \geq 1080$ using $Euler_1$ and the Lemma. However I am getting different (and slightly worse) results than your $N \geq 296,136$ for the use of the Lemma with $Euler_2, Euler_3$; will report on the thresholds I get soon (and then I will also work on confirming $Euler_5$). Looking at your code, I think the problem is that one shouldn't be putting absolute values around the cond[n][] terms in lines 221, 222 of ab_analysis.py; the signs of bn2, an2, etc. are actually rather important in the Lemma. By the way I am also putting the lemma and the numerical results on the wiki page.
EDIT: For $Euler_2 + Lemma$ I get $N \geq 341$ (replacing $N \geq 296$). For $Euler_3 + Lemma$ I get $N \geq 220$ (replacing $N \geq 136$).
EDIT2: $N \geq 282$ threshold for $Euler_5 + Triangle$ confirmed. For $Euler5 + Lemma$ I get $N \geq 241$ (i.e. worse than $Euler3 + Lemma$)!
EDIT3: I can improve the $N \geq 220$ threshold for $Euler3 + Lemma$ slightly to $N \geq 213$ by dropping the $d = 6$ term from the $Euler3$ mollifier, i.e. taking $\lambda_1 = 1, \lambda_2 = -b_2, \lambda_3 = -b_3$ and all other terms zero.

> **KM** · 27 February 2018
>
> Thanks, I removed the absolute part around the cond terms. There was another typo where i was using bn and an to compute bn2,an2,bn3,..., thus multiplying two conditional terms, instead of treating them independently.
> I was able to match N=341 for E2 + Lemma and N=220 for E3+ Lemma. But I am getting N=192 for E5 + Lemma (at N=241, the estimate is coming out to be 0.888)
> Using the general procedure with the sharper inequality, I get N=180 for E7+Lemma, and 176 for E11 + Lemma
> I have also started a x point evaluation run for x between 200k and 600k (N=219) with an adaptive mesh.
>
> > **Terence Tao** · 27 February 2018
> >
> > Ah, I made a stupid typo in my own code. I can now confirm your value of $N = 192$ for $E_5 + Lemma$.
>
> **KM** · 27 February 2018
>
> I wasn't able to replicate N=213 with the custom mollifier $\lambda_1 = 1, \lambda_2 = -b_2, \lambda_3 = -b_3$, and instead got N=235. For now, I have added a function abtoy_custom_mollifier in ab_analysis.py, and will experiment again later with different mollifiers.
>
> > **Terence Tao** · 27 February 2018
> >
> > Gah, you're right; the same typo that caused an error in my previous calculation was also present here. I've confirmed your value of 235. So I guess this is not a good mollifier :)
>
> **KM** · 1 March 2018
>
> Custom mollifiers do work. By treating the lambda coefficients as parameters to be optimized, and minimizing the Lemma sum using a standard minimization algorithm (the sum for D=30 wasn't fully minimized), we get
> $D = 2, \lambda_1 = 1, \lambda_2 = -1.15 \Rightarrow N = 336, \sum = 0.9995$
> $(1.15 \text{ is close to } \pi/e)$

$D = 6, \lambda_1 = 1, \lambda_2 = -1.045, \lambda_3 = -1.12, \lambda_6 = 0.9135 \Rightarrow N = 212, \sum = 0.997$
$D = 30, \lambda_1 = 1, \lambda_2 = -1.1455, \lambda_3 = -1.1189, \lambda_5 = -1.2779, \lambda_6 = 1.0447, \lambda_{10} = 1.1377, \lambda_{15} = 1.5231, \lambda_{30} = -1.6408 \Rightarrow N = 182, \sum = 0.998$



Terence Tao · 1 March 2018

Great! I've added this to the wiki (now moved to http://michaelnielsen.org/polymath1/index.php?title=Controlling_A%2BB/B_0 because the previous page http://michaelnielsen.org/polymath1/index.php?title=Polymath15_test_problem was becoming slow to load). It seems that trying to optimise the mollifier shrinks the range of x that we need to check by about 10-20% which is not that dramatic, but perhaps these sorts of optimisations add up (or more precisely, multiply up) after a while...



Anonymous · 26 February 2018

I can't find the code that participants are using for this project, the github linked from the project wiki hasn't had any commits in almost two weeks.



KM · 26 February 2018

Actually, all the new code is in the adjusted AB branch which has been in merge review since couple of weeks. Have merged that and Prof. Tao's recent branch (with Maple code and relevant papers) into master, which is much easier to find.

In the python folder, please check mputility.py and the research folder.



Rudolph · 27 February 2018

I acknowledge that the main task now is to establish the lowest possible x-bound beyond which $H_{eff,t=0.4}(2(x+0.4i)) \neq 0$. The encouraging results of the toy-model indicate that this x-bound is going to be brought well in reach of computing that $H_{eff,t=0.4}(2(x+0.4i))$ (and $y \geq 0.4$) doesn't vanish below that. In the context of computation, I would like to share the outcomes of an experiment I did.

In a nutshell: an x-ray plot shows all the points where $\Re\big(H_t(2(x+yi))\big) = 0$ (green lines) and $\Im\big(H_t(2(x+yi))\big) = 0$ (red lines), so the H_t-function only vanishes when the two lines cross. For a certain fixed y, a real line (that wiggles quite a bit) is always accompanied by two imaginary lines (which are much better behaved). Hence the idea to measure the distance a green line keeps from its embracing red lines and then to calculate the distribution of these distances over a wide range of x.

The approach is visualised here: https://ibb.co/jTihiH

The distributions for various t over x ranging from 1000 to 50000 (yielding 62870 $\Re\big(H_t(z)\big) = 0$'s), for both $y = 0$ and $y = 0.4$ is shown here: https://ibb.co/m1cExc

Few observations:

1) The line $\Im\big(H_t(2(x+0i))\big) = 0$ doesn't oscillate, hence to embrace $\Re\big(H_t(2(x+0i))\big) = 0$ I used $\Im\big(H_t(2(x+0.1i))\big) = 0$.

2) I appreciate that the Ht-ABC-formula has been deprecated now, however since we don't have an effectively bounded H_{eff} C-error term for lower x yet, I still used it to produce the plots (also since the match with $\Xi(t)$ at lower x was pretty good).

3) The data looks encouraging for the problem at hand. Distributions clearly tend to convergence towards more 'normal' shapes with increasingly emptier 'tails', when t and y move away from zero.

4) I wonder whether it is correct to say that when the distribution for $y = t = 0.4$ has empty tails say below 40% and above 60%, that given certain error terms of say max 5%, that line then must be zero-free up to $x = 50.000$?

Couldn't resist a philosophical note :)

The paraphrased Newman-quote: 'RH if true, then it is barely so' is also illustrated by the plots.

When t and y move towards zero, the distribution gets increasingly wider. Assuming the RH ($\Lambda = 0$), the non-trivial zeros apparently need to get 'crammed' into all the available slots at $t = y = 0$ to be able to encode the required information about the error term in the distribution of primes. The zeros seem to dangerously balance on the thin edge of a real cliff and even the tiniest negative t would already push some of them over into the sea of complex numbers...

Anonymous · 27 February 2018

Hah! That Newnan quote is 'barely' rubbish.. And RH is true! 😄

Terence Tao · 27 February 2018

Nice pictures! One small thing: I think the notational conventions we've been using are to use $z = x + yi = x + 0.4i$ for the argument of H_t, and $s = \frac{1+iz}{2} = 0.3 + \frac{ix}{2} = \sigma + iT$ for the argument of ξ_t (thus for instance $T = x/2$). It seems your conventions may differ from this by a factor of two, although for the purposes of qualitative illustration this probably doesn't matter.

Rudolph · 28 February 2018

You are absolutely right and I should have been much clearer on why I used $2z$ as the argument! I actually did start from the convention that $H_t(z) \sim \frac{1}{8}\xi_t(\frac{1}{2} + \frac{iz}{2})$ with $z = x + iy$, however then consciously multiplied z by 2 to obtain $H_t(2z) \sim \frac{1}{8}\xi_t(\frac{1}{2} + iz)$ that allows for better (visual) comparison to known values (zeros) of $\xi_0(\frac{1}{2} + iz)$ (for instance the sample x-ray in the first picture is from the area where the first Lehmer pair did induce some still visible 'wiggles' in the green lines).

Terence Tao · 28 February 2018

In that case your y=0.4 may in fact correspond to what the rest of us would call y=0.8 (but I am not 100% sure on this, perhaps one can crosscheck against some of the numerical data in http://michaelnielsen.org/polymath1/index.php?title=Polymath15_test_problem#Choices_of_approximation , which is for y=0.4 in our notation).

Terence Tao · 27 February 2018

p.s. I have now updated the wiki page at http://michaelnielsen.org/polymath1/index.php?title=Polymath15_test_problem#Choices_of_approximation with the refined approximation $A^{eff} + B^{eff} - C^{eff}$ to $A^{eff} + B^{eff}$. In practice it seems that C^{eff} stays quite close to C (and $A^{eff} + B^{eff} - C^{eff}$ stays close to $A + B - C$).

David Bernier (@doubledeckerpot) · 28 February 2018

This is my first try at translating into PARI/gp.
It gives H_0(28.2073) ~= 0 , which seems quite good.

```
Heff2(Z)=A(Z)+B(Z)-C(Z)
C(Z)=(1/8)*exp(t*(Pi^2)/64)*(sp(Z)*(sp(Z)-1)/2)*((-1)^M(Z))*(Pi^(-
sp(Z)/2)*gamma(sp(Z)/2)*(a(Z)^(-sig(Z)))*C0(Z)*U(Z)+Pi^(-(1-sp(Z))/2)*gamma((1-
sp(Z))/2)*(a(Z)^(-(1-sig(Z))))*conj(C0(Z))*conj(U(Z)))
sp(Z)=(1-imag(Z))/2+I*real(Z)/2+I*Pi*t/8
a(Z)=sqrt((real(Z)/2+Pi*t/8)/(2*Pi))
sig(Z)=real(sp(Z))
C0(Z)=(exp(Pi*I*((p(Z)^2)/2+3/8))-I*sqrt(2)*cos(Pi*I/2))/(2*cos(Pi*p(Z)))
U(Z)=exp(-I*((Tp(Z)/2)*log(Tp(Z)/(2*Pi))-Tp(Z)/2-Pi/8))
Tp(Z)=real(Z)/2+Pi*t/8
p(Z)=1-2*(a(Z)-M(Z))
```

M(Z)=floor(sqrt((real(Z)/2+Pi*t/8)/(2*Pi)))
A(Z)=(1/8)*exp((t/4)*(al1(I*real(Z)/2+(1-imag(Z))/2)^2))*H01(I*real(Z)/2+(1-imag(Z))/2)*sum(Y=1,M(Z),1/exp((I*real(Z)/2+(1-imag(Z))/2+t*al1(I*real(Z)/2+(1-imag(Z))/2)/2-(t/4)*log(Y))*log(Y)))
B(Z)=(1/8)*exp((t/4)*conj(al1(I*real(Z)/2+(1+imag(Z))/2))^2)*conj(H01(I*real(Z)/2+(1+imag(Z))/2))*sum(Y=1,M(Z),exp((-I*real(Z)/2+(1+imag(Z))/2+conj(t*al1(I*real(Z)/2+(1+imag(Z))/2))/2-(t/4)*log(Y))*log(Y)))
H01(S)=(S*(S-1)/2)*exp((-S/2)*log(Pi))*sqrt(2*Pi)*exp((S/2-1/2)*log(S/2)-S/2)
al1(S)=1/(2*S)+1/(S-1)+(1/2)*log(S/(2*Pi))
B0(Z)=(1/8)*exp((t/4)*conj(al1(I*real(Z)/2+(1+imag(Z))/2))^2)*conj(H01(I*real(Z)/2+(1+imag(Z))/2))
====
? real(Heff2(28.207303425056217036624775835617549798))
= 1.6860423122756198997 E-41
? t
= 0.E-38



Terence Tao · 28 February 2018

Thanks for the confirmation! That is remarkably good accuracy for such a small value of x. I wonder if it would be possible to have some comparative tables or plots of H_t, $A+B$, $A+B-C$, $A^{eff}+B^{eff}$, and $A^{eff}+B^{eff}-C^{eff}$ for such values of x (perhaps normalised by dividing by something like B_0^{eff} to eliminate the exponential decay). It may help give a sense of when the $A^{eff}+B^{eff}$ approximation is no longer viable, in which case one would need to switch to $A^{eff}+B^{eff}-C^{eff}$ (and get better explicit error terms).



David Bernier (@doubledeckerpot) · 28 February 2018

I think I can produce that normalized data (after dividing by B_0^{eff}. Indeed, 28.207 is pretty close to twice 14.13 (first non-trivial zeta zero).
The computer is working on this:
I'm computing the absolute value of $(H_{0.4}(z)-H_{0.4}^{eff}(z))/B_0^{eff}(z)$ for 20 real numbers z from 160 to 3200. Here, H^{eff} includes the C^{eff} term, and $H_{0.4}(z)$ is computed using the integral formula defining H_t .
T.B.C. ...



David Bernier (@doubledeckerpot) · 28 February 2018

[continued...]
There seems to be a local maximum once again near $z=800$.
The PARI/gp code to do the comparison of $H_{0.4}^{eff}$ with $H_{0.4}$, with the output, is copied below:
? \p 550
realprecision = 558 significant digits (550 digits displayed)
? w20=vector(20)
= [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
? t=0.4
[...]
? intnumgaussinit(800);
[...]
? for(K=1, 20, x50= K*160 ;s=0.0;for(J=0,30,s=s+intnumgauss(X=J/13,(J+1)/13, exp(t*X*X)*Phi(X)*cos((x50)*X),)); s2=Heff2(x50); b0=B0(x50);s4 = abs((s2-s)/b0);w20[K] = s4;)

? \p 10
realprecision = 19 significant digits (10 digits displayed)
? for(K=1,20, print(K*160," ", w20[K]))
160 0.009155667752
320 0.0005529962481
480 0.0004966282128
640 0.0004482768972
800 0.002686105466 // local maximum
960 0.001270168744
1120 0.0009957229500
1280 0.0007024411378
1440 0.0007000473085
1600 0.0004882487218
1760 0.0002518910919
1920 0.0008378989413
2080 0.0004924765754
2240 0.0001171320991
2400 0.0002443802551
2560 0.0004569058755
2720 0.0006355966221
2880 0.0008864917365
3040 4.326840265 E-5
3200 0.0003598521453



Terence Tao · 28 February 2018

Thanks for this data! I put it on the wiki at http://michaelnielsen.org/polymath1/index.php?title=Polymath15_test_problem#Controlling_.7CH_t-A-B.7C.2F.7CB_0.7C next to your previous data on $|H_t - (A+B-C)|/|B_0|$ (see third table) and the improvement in accuracy is substantial. Could you also give data for $|H_t - (A+B)|/|B_0|$ and $\left|H_t - (A^{eff}+B^{eff})\right|/\left|B_0^{eff}\right|$ for the same range of x? This would give some guidance as to the range of x for which we would have to utilise the C correction.



David Bernier (@doubledeckerpot) · 28 February 2018

Below, for the sake of getting results faster, I take $H_{0.4}^{eff}$ as the "standard" to which $A^{eff}(z)+B^{eff}(z)$ is compared, after division by B_0^{eff} :
? QABeff(Z) = abs(A(Z) + B(Z) − Heff2(Z))/abs(B0(Z))
? for(K=3,7, print(10^K," ", QABeff(10^K)))
1000 0.15607
10000 0.033581
100000 0.0077278
1000000 0.0017772
10000000 0.00026013



Terence Tao · 28 February 2018

Is your H^{eff} the same as $A^{eff}+B^{eff}-C^{eff}$? If so, then I guess your $QABeff$ would be $\left|C^{eff}\right|/\left|B_0^{eff}\right|$. I get very slightly different data for this ratio (here I am assuming $y=0$):
1000 0.1659471018
10000 0.03571919360

100000 0.008219920842
1000000 0.001890567784
10000000 0.0002777949355

The differences are small, but I'd like to pin it down – probably there is a typo in my own code since it seems your implementation approximates H_t to very high accuracy. For $z = 10^3$, for instance, I am getting these values:

$A^{eff} = -0.5819206433 \times 10^{-167} - 0.303931888110^{-167}i$
$B^{eff} = -0.5819206433 \times 10^{-167} + 0.303931888110^{-167}i$
$C^{eff} = -0.5290455851 \times 10^{-167}$
$B_0^{eff} = 0.1839421902 \times 10^{-166} + 0.2603864477 \times 10^{-166}i$

Does this match your numbers?



David Bernier (@doubledeckerpot) · 28 February 2018

This is sample code to write to a file the normalized values of $A^{eff} + B^{eff}$, $H_{0.4}^{eff}$:
? for(K=20, 1000, x50=K; w = (A(x50)+B(x50))/B0(x50); write(outputHeffAB, x50," ", w))
? for(K=20, 1000, x50=K; w = Heff2(x50)/B0(x50); write(outputHeffABC, x50," ", w))
But I don't know how to do plots in PARI/gp .



Rudolph · 28 February 2018

For z^3, t=0.4 I am currently getting (hoping my code is correct...):
A_eff -5.8191933398261330947 E-168 – 3.0393189395893835125 E-167*I
B_eff -5.8191933398261330947 E-168 + 3.0393189395893835125 E-167*I
C_eff -4.9757097161525819004 E-168 + 0.E-205*I
B0_eff 1.8394229213245936177 E-167 + 2.6038636181013073063 E-167*I

Just to show the good match I get between the ξ-function (1st) with (A+B-C)_eff. C-term listed independently (3rd) at t=0, y=0:

x=25
0.00066948088996085741409 – 7.174648137343063403 E-42*I
0.000657342156788652042 59 + 0.E-41*I
0.00021485873745531089 88
x=50
1.7280715206373737196 E-9 – 1.9158377441940858392 E-47*I
1.6625508526575364227 E-9 + 0.E-45*I
7.4751288517910783610 E-8 + 0.E-45*I
x=100
3.9527439074473614238 E-16 – 1.1093017595597491695 E-53*I
3.9779357479763186216 E-16 + 0.E-53*I
-4.0160884429998658723 E-16 + 0.E-53*I
x=500
1.0763588275034730133 E-82 + 5.653984195680004678 E-119*I
1.0770837316884739789 E-82 + 0.E-120*I
-2.1026699008978412660 E-83 + 0.E-120*I
x=1000
-3.0912467355888789916 E-167 – 3.251797352795581408 E-203*I
-3.0904791459693381129 E-167 + 0.E-205*I
-5.2704332443772292555 E-168 + 0.E-205*I
x=10000
1.1326355002579265086 E-1700 + 2.3068258704312139077 E-1736*I

1.1324594118594644316 E-1700 + 0.E-1738*I
-1.4441315742576007239 E-1701 + 0.E-1738*I

 **Anonymous** · 28 February 2018

I can confirm the full precision of Rudolph's values for A^{eff}, B^{eff}, and B_0^{eff} using my own implementation based on the wiki for $x = 1000y = 0t = 0.4$, but I haven't yet implemented the C^{eff} correction.

 **David Bernier (@doubledeckerpot)** · 28 February 2018

Yes, it should all be the effective type, unless I'm mistaken.
I seem to get the same thing as you for B_0^{eff}, slightly different w.r.t. A^{eff} and B^{eff}, and really different for C^{eff}:
? x50=1000; w = A(x50); print(w)
-5.819193340 E-168 – 3.039318940 E-167*I
? x50=1000; w = B(x50); print(w)
-5.819193340 E-168 + 3.039318940 E-167*I
? x50=1000; w = C(x50); print(w)
-4.975709716 E-168 + 0.E-186*I
? x50=1000; w = B0(x50); print(w)
1.839422921 E-167 + 2.603863618 E-167*I

 **Terence Tao** · 28 February 2018

Ah, thanks for this, I tracked down the bugs in my own code. (For the record: I had used $T = x/2$ instead of $T' = x/2 + \pi t/8$, and I had also omitted the factor of $\exp(t\pi^2/64)$.) My data now matches yours to three significant figures; for instance I have $C^{eff} = -4.97671022 \times 10^{-167}$ now. The discrepancy is a lot smaller but I'd still like to pin down what's going on; as far as I can tell it is not simply a matter of not having enough digits of precision. Staying with $x = 1000, y = 0, t = 0.4$, I also calculate
$C_0(p) = 0.3763483070 - 0.2292780192i$
$U = -0.5779312326 - 0.816085467i$
$\Gamma(s'/2) = 1.333548739 \times 10^{-171} - 8.4934750812 \times 10^{-172}i.$
Do these correspond to the numbers you are getting?

 **Rudolph** · 28 February 2018

I got:
Co(p) 0.376348307023843 + 1.58717855690832*I
U -0.577931232612849 – 0.816085467564882*I
Gamma(s'/2) 1.33354873965418 E-171 – 8.49347508127889 E-172*I
So, the difference seems to be only in the imaginary part of C0(p). The error could easily be in my code, but here is a thought: for C0 my code is:
C0(s)=(exp(Pi*I*((P(s)^2)/2+3/8))-I*sqrt(2)*cos(Pi*I/2))/(2*cos(Pi*P(s)))
I can simply put C0(s) since the only variable is P(s), so I will always get p as the outcome. When I would put in C0(P(s)), it would give a very different outcome. I think that somewhere in this space the difference originates from.

 **Anonymous** · 28 February 2018

Again I get the same numbers as Rudolph for those three parts of C^{eff} when I use the formulas on the wiki.

C0 : 0.37634830702384258887 + 1.5871785569083238623*I
U : -0.57793123261284940132 – 0.81608546756488229758*I
Gamma(s' / 2) : 1.3335487396541778349e-171 – 8.4934750812788891334e-172*I

Anonymous · 28 February 2018

There's a typo in the wiki where $\cos(\pi i/2)$ should be $\cos(\pi p/2)$. With this correction I get imaginary part 0.22927801920219723181 matching Terry's number.

Anonymous · 28 February 2018

imaginary part -0.22927801920219723181 I mean, with the negative sign. The typo is in the last part of the numerator of the definition of $C_0(p)$ in http://michaelnielsen.org/polymath1/index.php?title=Polymath15_test_problem#Choices_of_approximation.

Terence Tao · 1 March 2018

Thanks for this! I'm glad now that we tracked down the numerical discrepancy, we might not have discovered the typo for a long time otherwise. I've corrected the wiki.

David Bernier (@doubledeckerpot) · 28 February 2018

Right now, the computer is working on the data for the table: z = 160 to 3200, $H_{0.4}(z)$ (integral) compared to $A(z) + B(z)$ ...

Rudolph · 28 February 2018

Good catch, that must be it.
One more question: when we 'call' C0 from the formula for C, should we use s or s' ? It is only a small difference, but just be sure.

Terence Tao · 1 March 2018

Well, the way it is done in the wiki, C_0 is a function of p, which is in turn a function of a and N, which are functions of T', which is the imaginary part of s', which is a shifted form of s. So one could make C_0 a function of p, or of T', or of s', or of s, depending on how one codes it.

Anonymous · 28 February 2018

Sorry Rudolph I can't answer your questions about "when we 'call' C0 from the formula for C, should we use s or s'" or "I can simply put C0(s) since the only variable is P(s), so I will always get p as the outcome". Maybe they're technical questions about how to use Pari/GP but I'm not using that system and I haven't used it.

Anonymous · 28 February 2018

For C^{eff} I see -4.9767102201547040018e-168 which is off by exactly a factor of 10 from what Terry wrote, but I assume he just typo'd 167 instead of 168 when writing his comment.
[Yes, that was a typo on my part – T.]

Rudolph · 28 February 2018

Also got the code for new C working and the match at t=0 seems excellent even at small x (can confirm David's zero at 28.207). Could there be a small typo on Wiki where C0(p) should actually be C0(s) (also for its conjugated version)?

Terence Tao · 28 February 2018

It should indeed be $C_0(p)$ (though, as p ultimately depends on s, one could also view $C_0(p)$ as a function of s). Here I am following the notation of the Arias de Reyna paper https://pdfs.semanticscholar.org/7964/fbdc0caeec0a41304deb8d2d8b2e2be639ee.pdf (see e.g. equation (5.2)).

Rudolph · 28 February 2018

You're right! It should be $C_0(p)$ indeed, apologies for the false alarm.

Rudolph · 1 March 2018

A final question just to be sure that I have translated the wiki-formulae correctly into computer code.
On wiki, the definition of C^{eff} uses s' as the main variable in the arguments of all its components. Since p ultimately is a function of s, should the p in the component $C_o(p)$ be dependent on s or also on s' ? For U the same question could be raised.
For $x = 1000, y = 0, t = 0.4$, the choice would give this difference in C^{eff}:
using $C_0(s) \to -4.97671022015470E-168+0.E-186*I$
using $C_0(s') \to -4.99500582650853E-168+0.E-186*I$

Terence Tao · 1 March 2018

From my reading of David's code, it seems that p is a function of s (which is called Z in the PARI code) rather than s' (as the shift by $\pi T/8$ is coded into these functions), so one would use s as the input for David's C_0 function rather than s'.

Anonymous · 28 February 2018

I don't see the definition of σ in C^{eff}, is it $(1-y)/2$?
[Yes, I have updated the wiki to reflect this – T.]

Anonymous · 27 February 2018

This is probably a dumb question but are there explicit tail bounds for the E3 improper integral error term?

Terence Tao · 27 February 2018

Not yet; that's something I plan to look at soon, as it looks like E_3 is the dominant error and also the one which is hardest to estimate at present.

anonymous · 28 February 2018

It's not clear to me what the optimal way of bounding E_3 will be, but one simple-minded way is just to truncate the integral defining it at some large values of σ, relying on the decay of $f(\sigma)$ and the not-too-fast increase of w, v to show that outside this truncation the integral is small, while inside the truncation $w(\sigma)$ is close to 1, and otherwise evaluate everything algebraically as far as possible.
This is complicated a little bit by discontinuities in the definition of v, but I think these should be able to be dealt with in one way or another.
I've only gotten as far as to estimate w inside some hypothetical cutoffs, and even then I've not made an effort to optimize these cutoffs and just done a sort of back-of-the-envelope computation with _a lot_ of lost information in the constants. Nonetheless, it seems that something like the following will be true:

For $T_0' > 20$ say, for $\sigma \in (0, \sqrt{T_0'})$,
$w(\sigma) = 1 + O_{\leq}\left(\frac{3}{2}\frac{1}{T_0'}\right)$,
while for $\sigma \in (-(T_0')^{1/3}, 0)$, the same bound holds true.
Haven't yet plugged these into the integrals or dealt with the range outside this truncation...



Terence Tao · 28 February 2018

This is a promising approach! I will try it and see what I get.



Anonymous · 27 February 2018

On the wiki page for this test problem, should the second $B^{eff} :=$ be $B_0^{eff} :=$?
[Corrected, thanks – T.]



Anonymous · 27 February 2018

I think there's a sign typo on this thread's wiki, where $\frac{1+y-ix}{2}$ in part of the definition of B^{eff} should be $\frac{1+y+ix}{2}$.



Anonymous · 27 February 2018

If the sign is correct, then there is a corresponding typo in the effective bounds wiki page; the signs are inconsistent between these two pages.



Terence Tao · 27 February 2018

It was the effective wiki page that had the sign error, now corrected; thanks for pointing it out.



Terence Tao · 27 February 2018

I have entered some calculations on the wiki on how to transition from the toy model to the effective approximation.
Currently, to show that $A^{toy} + B^{toy}$ is non-vanishing, we have to verify that
$$\left|\sum_{n=1}^{N} \frac{b_n}{n^{s^{toy}}}\right| - \left|\sum_{n=1}^{N} \frac{a_n^{toy}}{n^{s^{toy}}}\right| > 0$$
where $s^{toy} := \sigma^{toy} + iT^{toy}$ has real part $\sigma^{toy} := \frac{1+y}{2} + \frac{t}{2}\log N$ (and imaginary part $T^{toy} := \frac{x}{2} + \frac{\pi t}{8}$, though this imaginary part has not played much role so far), $b_n := \exp(\frac{t}{4}\log^2 n)$, $a_n := (n/N)^y b_n$, and we took $N := \lfloor\sqrt{\frac{x}{4\pi}}\rfloor$. We now have a number of ways to establish this lower bound for various values of N.
For the problem of showing that $A^{eff} + B^{eff}$ is non-vanishing, it similarly suffices to verify that
$$\left|\sum_{n=1}^{N} \frac{b_n}{n^{s^{eff}}}\right| - \left|\sum_{n=1}^{N} \frac{a_n^{eff}}{n^{s^{eff}}}\right| > 0 \quad (1)$$
where $s^{eff} = \sigma^{eff} + iT^{eff}$ has a more complicated form than s^{toy}, but the real part σ^{eff} is greater than or equal to σ^{toy} (assuming $y \geq 1/3$, which we have right now), b_n is as before, N is very slightly different as $N := \left\lfloor\sqrt{\frac{x}{4\pi} + \frac{t}{16}}\right\rfloor$ (thus $x_N \leq x < x_{N+1}$ where $x_N := 4\pi N^2 - \frac{\pi t}{4}$), and a_n^{eff} turns out to take the form
$a_n^{eff} = e^{\delta} a_n^{toy} + O_{\leq}(e_n)$
where δ is the small quantity
$$\delta := \frac{\pi y}{2(x_N - 6 - \frac{14+2y}{\pi})} + \frac{2y(7+y)}{x_N^2}\log\frac{|1+y+ix_{N+1}|}{4\pi}$$
(roughly of size $O(1/x)$) and e_n is a small error
$$e_n := e^{\delta}(\exp(\frac{ty\log n}{2(x_N - 6)} - 1)a_n^{toy}.$$

(roughly of size $O(a_n^{toy} \frac{\log n}{x})$). So, up to the δ and e_n factors, a_n^{eff} is basically the same as a_n^{toy}, and the previous analysis should go through (the fact that σ^{eff} is slightly larger than σ^{toy} actually works in one's favor, one can simply just use the criteria we have with just σ^{toy}). The one annoying thing is that the error term $O_{\leq}(e_n)$ is complex valued, and the Lemma that has been giving the best values of N requires real coefficients. So one basically has to extract out the e_n errors separately using the triangle inequality, leading to a messier criterion for showing non-vanishing (or for more generally lower bounding the LHS of (1)). In principle one can also insert the Euler product mollifiers as before; the criterion then gets somewhat messy but should still be usable.

KM · 28 February 2018

For the effective approximation with the Lemma inequality, the same bounds as the current toy bounds seem to hold (I checked with upto 2 Euler factors) – 1080, 341, 220. Away from the critical N value we get different estimates. For e.g. at N=100, |b2| – |a2| toy estimate is 1.612 and effective estimate is 1.515, but they both go below 1 at N=341.
Also, I stopped the adaptive mesh x point toy evaluation at 500k (N=199) (which was started at x=200k). Mesh size was determined as (|b2(x)|-|a2(x)|)/4. Minimum value for |b2(x)|-|a2(x)| was 0.604. Hence with the earlier exercise upto 200k, the entire range 10k to 500k has been now considered. I will now evaluate the effective approximation for the same range.

KM · 28 February 2018

Also some minor typos in that section of the wiki. The triangle inequality has the n^sigma factor missing, and in the Lemma inequality, 1-a1 and 1+a1 should be flipped.



Terence Tao · 28 February 2018

Good catches! I have corrected the wiki accordingly.

Anonymous · 27 February 2018

In the "Final bound" section of the effective bounds wiki page and in the effective bounds section of the test problem page, some of the error terms involve a T. Should these be T' or T_0 or does the T mean $x/2$ or something else?
[T is $x/2$; I have added this to the description. -T.]



David Bernier (@doubledeckerpot) · 28 February 2018

For the range 160 to 3200, I can provide data for tables, after a break :)



David Bernier (@doubledeckerpot) · 28 February 2018

This is what I get for $(H_{0.4}(z) - (A^{eff}(z) + B^{eff}(z)))/B_0^{eff}(z)$ in absolute value:

```
? for(K=1, 20, x50= K*160 ;s=0.0;for(J=0,30,s=s+intnumgauss(X=J/13,(J+1)/13,
exp(t*X*X)*Phi(X)*cos((x50)*X),)); s2=A(x50)+B(x50); b0=B0(x50);s4 = abs((s2-s)/b0);w20[K] = s4;)
? \p 19
 realprecision = 19 significant digits
? for(K=1,20, print(K*160," ", w20[K]))
 160 0.1675083979955609185
 320 0.2776948344513698276
 480 0.1675667240356922231
 640 0.1635077306008453928
 800 0.2045038601879677257
 960 0.1031837988358064657
 1120 0.07541968034203085865
```

1280 0.1339118061014743863
1440 0.07958929988050262854
1600 0.07700542235140914608
1760 0.09568042045936396570
1920 0.06385275621986742745
2080 0.09421231232885752514
2240 0.05888587520703223358
2400 0.07899341548208345822
2560 0.06283843631123482445
2720 0.05966972584730198272
2880 0.07980560515423855917
3040 0.04636072344121703969
3200 0.09664223832561922043



Terence Tao · 1 March 2018

Just to clarify: is this the same calculation as in your preceding comment https://terrytao.wordpress.com/2018/02/24/polymath15-fourth-thread-closing-in-on-the-test-problem/#comment-493282 , but with the corrected version of C^{eff}? The accuracy seems to be somewhat worse with this new set of data.



David Bernier (@doubledeckerpot) · 1 March 2018

No, it's a different calculation: it's the normalized error in $A^{eff} + B^{eff}$, as an approximation to $H_{0.4}$ (using the defining integral).

As far as I can tell, the corrected C_0 and C^{eff} do not affect values of $A^{eff} + B^{eff}$ ...



Terence Tao · 1 March 2018

Thanks for clarifying! I've added this to the table, which is now moved to http://michaelnielsen.org/polymath1/index.php?title=Controlling_H_t-A-B/B_0 because the page http://michaelnielsen.org/polymath1/index.php?title=Polymath15_test_problem was becoming slow to load. It is now quite clear that the C corrections do give a noticeable improvement in the accuracy of the approximation.



David Bernier (@doubledeckerpot) · 1 March 2018

One column is just a bit off, I think.

With the corrected C^{eff} , the normalized errors of $H^{eff}(z) - (A^{eff}(z) + B^{eff}(z) - C^{eff}(z))$ compared to the integral formula are likely a bit smaller.

For example, with the corrected C^{eff} , for z=1600, I now get a normalized error of 0.000439823 (versus 0.000488248 on the wiki).

Except for z = 800, the normalized errors on the wiki used the old C^{eff}: the one with a typo. I'm recalculating.

? x50= 1600 ;s=0.0;for(J=0,30,s=s+intnumgauss(X=J/13,(J+1)/13, exp(t*X*X)*Phi(X)*cos((x50)*X),)); s2=Heff2(x50); b0=B0(x50);s4 = abs((s2-s)/b0); s4 = 0.0004398235678



David Bernier (@doubledeckerpot) · 1 March 2018

Using the corrected C^{eff}, the normalized error in H^{eff} looks marginally better, on average, than when using the C^{eff} with a typo :

? for(K=1, 20, x50= K*160 ;s=0.0;for(J=0,30,s=s+intnumgauss(X=J/13,(J+1)/13, exp(t*X*X)*Phi(X)*cos((x50)*X),)); s2=Heff2(x50); b0=B0(x50);s4 = abs((s2-s)/b0);w20[K]

= s4;) // corrected C^eff
[…]
? for(K=1,20, print(K*160," ", w20[K]))
160 0.00887362155217
320 0.000708716878236
480 0.000327585584191
640 0.000523969818792
800 0.00264434457030
960 0.000819848578351
1120 0.000978838228690
1280 0.000679785836479
1440 0.000655447626435
1600 0.000439823567873
1760 0.000231103881282
1920 0.000849398936325
2080 0.000505410233739
2240 0.000107206342271
2400 0.000229146425813
2560 0.000492123116208
2720 0.000656180976718
2880 0.000878298302262
3040 4.70113907733 E-5
3200 0.000354582706466
[New data added, thanks – T.]



David Bernier (@doubledeckerpot) · 28 February 2018

My numbers are almost the same as Terry's: the $C_0(1000)$ values differ in their imaginary parts only, it seems… :
? C0(1000)
= 0.3763483070 + 1.5871785569*I // not the same
? U(1000)
= -0.5779312326 − 0.8160854676*I // the same
? gamma(sp(1000)/2)
= 1.3335487397 E-171 − 8.4934750813 E-172*I // the same



Anonymous · 28 February 2018

David there's a typo in the wiki (https://terrytao.wordpress.com/2018/02/24/polymath15-fourth-thread-closing-in-on-the-test-problem/#comment-493308) so
C0(Z)=(exp(Pi*I*((p(Z)^2)/2+3/8))-I*sqrt(2)*cos(Pi*I/2))/(2*cos(Pi*p(Z)))
should be changed to
C0(Z)=(exp(Pi*I*((p(Z)^2)/2+3/8))-I*sqrt(2)*cos(Pi*p(Z)/2))/(2*cos(Pi*p(Z)))



David Bernier (@doubledeckerpot) · 28 February 2018

I tested for comparison with $H_{0.4}(800)$ (integral formula).
With C0 "new", the error of $H^{eff}(800)$ is slightly smaller than
with C0 per the Wiki:
? C0new(Z) = (exp(Pi*I*((p(Z)^2)/2+3/8))-I*sqrt(2)*cos(Pi*p(Z)/2))/(2*cos(Pi*p(Z)))
? C0new(1000)
= 0.3763483070238425883 − 0.2292780192021972320*I

? C0(Z)=C0new(Z)
? x50= 800 ;s=0.0;for(J=0,30,s=s+intnumgauss(X=J/13,(J+1)/13,
exp(t*X*X)*Phi(X)*cos((x50)*X),)); s2=Heff2(x50); b0=B0(x50);s4 = abs((s2-s)/b0); s4
= 0.002644344570 (versus 0.002686105466 in Wiki table).



anonymous · 1 March 2018

When your main research reference is the first part of a seven year old two-part work that cites its second part as "to appear" https://i.imgur.com/IrSHTib.jpg



David Bernier (@doubledeckerpot) · 1 March 2018

After the wiki revision of C_0 now at: http://michaelnielsen.org/polymath1/index.php?title=Polymath15_test_problem#Controlling_.7CH_t-A-B.7C.2F.7CB_0.7C everything matches:

```
? C0(1000)
 = 0.3763483070238425883 - 0.2292780192021972320*I
? U(1000)
 = -0.5779312326128494048 - 0.8160854675648822951*I
? gamma(sp(1000)/2)
 = 1.333548739654177835 E-171 - 8.493475081278889133 E-172*I
========================
 user-defined functions :
A =
 (Z)-
>(1/8)*exp((t/4)*(al1(I*real(Z)/2+(1-imag(Z))/2)^2))*H01(I*real(Z)/2+(1-imag(Z))/2)*sum(Y=1,M(Z),1/
exp((I*real(Z)/2+(1-imag(Z))/2+t*al1(I*real(Z)/2+(1-imag(Z))/2)/2-(t/4)*log(Y))*log(Y)))
B =
 (Z)->(1/8)*exp((t/4)*conj(al1(I*real(Z)/2+(1+imag(Z))/2))^2)*conj(H01(I*real(Z)/2+(1+imag(Z))/2))*sum(Y=1,M(Z),1/
exp((-I*real(Z)/2+(1+imag(Z))/2+conj(t*al1(I*real(Z)/2+(1+imag(Z))/2))/2-(t/4)*log(Y))*log(Y)))
B0 =
 (Z)->(1/8)*exp((t/4)*conj(al1(I*real(Z)/2+(1+imag(Z))/2))^2)*conj(H01(I*real(Z)/2+(1+imag(Z))/2))
C =
 (Z)->(1/8)*exp(t*(Pi^2)/64)*(sp(Z)*(sp(Z)-1)/2)*((-1)^M(Z))*(Pi^(-sp(Z)/2)*gamma(sp(Z)/2)*(a(Z)^(-
sig(Z)))*C0(Z)*U(Z)+Pi^(-(1-sp(Z))/2)*gamma((1-sp(Z))/2)*(a(Z)^(-(1-sig(Z))))*conj(C0(Z))*conj(U(Z)))
C0 =
 (Z)->(exp(Pi*I*((p(Z)^2)/2+3/8))-I*sqrt(2)*cos(Pi*p(Z)/2))/(2*cos(Pi*p(Z)))
H01 =
 (S)->(S*(S-1)/2)*exp((-S/2)*log(Pi))*sqrt(2*Pi)*exp((S/2-1/2)*log(S/2)-S/2)
Heff2 =
 (Z)->A(Z)+B(Z)-C(Z)
M =
 (Z)->floor(sqrt((real(Z)/2+Pi*t/8)/(2*Pi)))
Tp =
 (Z)->real(Z)/2+Pi*t/8
U =
 (Z)->exp(-I*((Tp(Z)/2)*log(Tp(Z)/(2*Pi))-Tp(Z)/2-Pi/8))
a =
 (Z)->sqrt((real(Z)/2+Pi*t/8)/(2*Pi))
al1 =
 (S)->1/(2*S)+1/(S-1)+(1/2)*log(S/(2*Pi))
```

```
p =
 (Z)->1-2*(a(Z)-M(Z))
sig =
 (Z)->real(sp(Z))
sp =
 (Z)->(1-imag(Z))/2+I*real(Z)/2+I*Pi*t/8
```



Anonymous · 1 March 2018

Would it make sense to record that code in a new Pari/GP directory in the github project, next to the existing julia, maple, matlab, and python directories?



David Bernier (@doubledeckerpot) · 1 March 2018

I believe so, or maybe. It's my current best effort at coding the mathematical functions for the effective approximation, as written up by Terry on the Wiki. It doesn't have everything, but it has the definitions needed to calculate the effective approximations.



Terence Tao · 1 March 2018

Thanks for this! I think we can now be fairly confident in the accuracy of the formula for the refined effective approximation $A^{eff} + B^{eff} - C^{eff}$, thanks to everybody for helping out with the debugging process. I've also put a text file with David's PARI code into the git repository.



David Bernier (@doubledeckerpot) · 1 March 2018

This is what I get for $(H_{0.4}(z) - A(z) - B(z))/B_0(z)$, in the non-effective type estimates:

```
160 0.174873661533
 320 0.278624615745
 480 0.167598495339
 640 0.165084846603
 800 0.201954876756
 960 0.103387669714
 1120 0.0767779295558
 1280 0.132886551163
 1440 0.0802159981813
 1600 0.0777462698681
 1760 0.0950946156489
 1920 0.0629013452776
 2080 0.0949328843573
 2240 0.0591497767926
 2400 0.0785798163298
 2560 0.0621868667021
 2720 0.0585282736442
 2880 0.0787554869341
 3040 0.0462460274843
 3200 0.0963053589535
```

The PARI/gp session is still running, so we might cross-check a sample of $A(z) + B(z)$ values...

[Data added to wiki, thanks. Looks like the accuracy of $A + B$ and $A^{eff} + B^{eff}$ is almost identical – T.]



Terence Tao · 1 March 2018

I have tentatively calculated a fairly tractable and usable upper bound E_3^* for the main error term E_3 at the end of http://michaelnielsen.org/polymath1/index.php?title=Controlling_H_t-A-B/B_0 . Explicitly, the bound is

$E_3^* = \frac{1}{8}\sqrt{\pi}\exp(-\frac{t\pi^2}{64})(T_0')^{3/2}e^{-\pi T_0/4}\exp(\frac{0.181}{T_0'-3.33})(1+\frac{5.15}{a_0-1.25})$

where T_0 is any lower bound for T that is greater than or equal to 100, $T_0' := T_0 + \pi t/8$, and $a_0 := \sqrt{T_0'/2\pi}$. Actually one should just set $T_0 = T$ for the best bound (eventually I'll probably replace T_0 with T throughout in the wiki calculations in fact, it is a legacy from a period in which I was not sure if the error estimates were going to be monotone decreasing in T). I don't yet have the ability to easily calculate E_3 in maple, but using C^{eff} as a proxy it looks like E_3^* is only worse than $|C^{eff}|$ by a factor of two or so (see table at the end of the above link), and this looks promisingly small once x goes above 10^4 or so. Asymptotically, the last two factors in the above definition of E_3^* are close to 1, and the remaining expression $\frac{1}{8}\sqrt{\pi}\exp(-\frac{t\pi^2}{64})(T_0')^{3/2}e^{-\pi T_0/4}$ is basically the upper bound for $|C^{eff}|$ coming from the fact that $|C_0(p)| \leq 1/2$ (equality should occur when x is close to a discontinuity for N, i.e. $x + \pi t/4 \approx 4\pi N^2$).

In the end I didn't quite use the suggestion of splitting the σ integration into small and large portions; instead it was more efficient to estimate various factors in the integrands by expressions of the form $\exp(a\sigma^2 + b\sigma + c)$ for a not too large, which could then be absorbed into the gaussian factors in the f factor. The case of very large k had to be treated separately but here σ was very large and negative and the contribution was in fact super-exponentially small from this case.

I'll double check the E_3^* derivation soon, and also start working on getting some usable upper bound on E_1, E_2. If we have all that then it seems that by putting everything together we should be able to handle all x past a certain point (e.g. $N \geq 200$, which corresponds to $x \geq 5 \times 10^6$ or so), leaving one with a range that looks (barely) within the range of numerical verification.



Rudolph · 2 March 2018

I get an exact match with your wiki table, the one that compares the normalised C^{eff} and E_3^*. Also produced some graphs that illustrate how well the new bound actually works. The discontinuities that David spotted in the previous thread are well visible and appear to all stay within the bound.

Using $t = y = 0.4$:

For $500 \leq x \leq 10^3$: https://ibb.co/d7zDv7

For $500 \leq x \leq 10^4$: https://ibb.co/efY6F7

For $500 \leq x \leq 10^5$: https://ibb.co/eYdZ2n

For $500 \leq x \leq 10^6$: https://ibb.co/fjSu2n

For $500 \leq x \leq 10^7$: https://ibb.co/gDsf8S

Also got some encouraging timing results for the oncoming numerical verification effort. $H^{eff}(10^7 + 0.4i)$ with the latest C^{eff}, now computes in Pari/gp in 32.2 ms, compared to the previous C^{eff} (with the vwf-integral) that required 87.9 ms. A step change improvement :)

[Linked to on wiki, thanks! -T.]

arch1 · 2 March 2018

Thanks much for the graphs Rudolph. I see that one should be careful in drawing conclusions about the fine structure when eyeballing these. See e.g. x=[1200,10000] in graph 3, then in graph 2.

 **Rudolph** · 2 March 2018

Note that the graph's y-axis scaling reduces for higher x to be able to keep the differences visible. Eyeballing these graphs should just help reinforce the intuition and most certainly won't provide any hard evidence :)

 arch1 · 2 March 2018

Yup thanks again.

 **Anonymous** · 2 March 2018

Is it possible to derive (for any fixed positive t, y and sufficiently large x) a simple effective bound for $H_t(x+iy)/B_0^{eff} - 1$ which would decay to zero for large x?

It seems that such bound is sufficient to give an effective upper bound (i.e. an explicit function of t) on the real part of any nonreal zero of H_t.

 **Terence Tao** · 2 March 2018

We're close to doing this I think. By the triangle inequality we have

$$\left|\frac{H_t}{B_0^{eff}} - 1\right| \leq \frac{|A^{eff}|}{|B_0^{eff}|} + \frac{|B^{eff} - B_0^{eff}|}{|B_0^{eff}|} + \frac{|E_1|}{|B_0^{eff}|} + \frac{|E_2|}{|B_0^{eff}|} + \frac{|E_3^*|}{|B_0^{eff}|}.$$

All terms on the right hand side are known to go to zero as $x \to \infty$, and if we made the effort we could probably get enough effective bounds on all of these to produce an explicit rate. But this would probably not be the most efficient way to keep H_t away from zero; in particular it seems useful to multiply H_t/B_0^{eff} by a suitable mollifier first before subtracting 1 in order to reduce the oscillation.

 **Polymath15, fifth thread: finishing off the test problem? | What's new** · 2 March 2018

[...] thread of the Polymath15 project to upper bound the de Bruijn-Newman constant , continuing this post. Discussion of the project of a non-research nature can continue for now in the existing proposal [...]